A GUIDE FOR CAREER SWITCHERS

Nick's Guide to *AI*

Learn AI by doing it. A path from zero to AI Native, for anyone aiming at AIOM, Forward Deployed, or any applied-AI role.

by Harry Toig · AI Outcomes Manager at Glean

No prerequisites. No CS degree required. Four years ago I was in finance at Snowflake. Then I watched some YouTube videos, talked to a guy named Max Comolli, and started building. I am now an AI Outcomes Manager at Glean. The ladder is climbable.

> ***The thesis: learn AI by doing it. Not by reading about it. Not by grinding courses. Not by polishing your resume. Pay $20 a month for Claude, build five things, and be dangerous in six months.***

This guide targets a specific end state: **AIOM** (AI Outcomes Manager) or any adjacent applied-AI role: Forward Deployed Engineer, Solutions Engineer, AI Product Manager, Customer Engineer. These roles sit between the people building frontier models and the customers using them. They're the most gettable AI jobs for non-technical backgrounds.

The North Star: AI Native

Glean's internal framework for AI fluency has three tiers. The hiring bar for AIOM is the top one.

AI Literate	Uses ChatGPT competently. Writes decent prompts. Below the floor.
AI Proficient	Has built agents. Uses Claude Code. Has workflows that measurably change their output.
AI Native ←	Builds habitually. Architects workflows from scratch. Iterates prompts as craft. Debugs failure modes. Can't imagine working without these tools.

AI Native is not a credential. It is what you are after you have shipped enough things. You get there by building.

What You Need to Know

If you read one section in this guide, read this. The specific skills the AIOM path requires. The chapters that follow are the curriculum.

TECHNICAL

- **Prompting as craft.** *Role → Task → Context → Reasoning → Output → Stop Condition.* Iteration, not one-shot.
- **Agent design.** Retrieval, grounding, guardrails, tool calling, multi-step flows.
- **RAG fundamentals.** Explain retrieval-augmented generation to a non-technical Sales Director in 90 seconds.
- **LLM fundamentals.** Context windows, reasoning vs. fast mode, evals, why models hallucinate.
- **MCP, Custom Actions, APIs.** When to use each. Webhooks, REST, OAuth basics.
- **JSON literacy.** Read and edit configs without flinching.
- **Reading code.** Not writing Python from scratch. Reading it when an agent you're debugging spits out a stack trace.

NON-TECHNICAL

- **Customer discovery.** Turn ambiguous pain into a shippable agent spec.
- **Change management.** Champion frameworks, departmental rollouts, adoption tracking.
- **Writing.** Briefings, runbooks, how-to guides customers actually use.
- **Enablement delivery.** Train 50-200 people without putting them to sleep.
- **Executive presence.** You'll brief CHROs, CTOs, and CEOs on their own AI strategy.

MINDSET

Hungry, curious, consultative. You build things for yourself, not for the interview. Low ego, high agency. Resilience with ambiguity. Candidates who drop off when asked to "build or present something" self-select below the bar.

The Three Moves That Matter

In order. Not negotiable.

01 Understand it.

Spend ten hours with the right people explaining what an LLM actually is. Not hype videos. The real ones.

02 Use it every day.

Until you have gut feel for what these things can and can't do, no amount of theory will save you.

03 Build something.

Not a to-do app. Something that only exists because AI exists. The single highest-leverage thing you can do. Everything else is commentary.

If you are short on time, start with #3. You can reverse-engineer the other two from there.

The Guide

01 Fundamentals →
Videos and courses to actually understand how LLMs work.

02 Build →
Nine projects to ship, sorted by dependency.

03 Claude Code →
The daily-driver setup you should steal.

04 What to Follow →
Podcasts, blogs, and X accounts worth your time.

05 The Role

→

What AIOMs actually do, and what skills get you there.

Your First Hour

If you do nothing else this week, do these five things today.

1. **Pay for Claude.** $20 per month. Use it for everything: writing, summarizing, planning, thinking. You cannot learn AI on the free tier.
2. **Install Claude Code.** `npm install -g @anthropic-ai/claude-code` . Point it at your laptop. Ask it to do something useful.
3. **Watch** Karpathy's "Intro to Large Language Models" (one hour, free). The highest-density AI explainer that exists.
4. **Subscribe to** Simon Willison's blog. He reads the internet so you don't have to.
5. **Pick a project from** Build. Start today. Finish it this week. Ship it publicly.

CHAPTER 01

Fundamentals

How LLMs work, what to learn first, and what to skip.

You don't need to understand attention math. You do need a working mental model: what an LLM is, what it can and can't do, how prompts, context, tools, and agents fit together. That's what this chapter buys.

Core (watch these, ~5-6 hours)

1. **Andrej Karpathy, Intro to Large Language Models** (1 hr). The single best "what is an LLM" video ever made. Non-technical but substantive. Start here.
2. **Andrej Karpathy, Software Is Changing (Again)** (40 min). The Software 1.0 → 2.0 → 3.0 framing. Essential mental model for where this is going.
3. **Andrej Karpathy, "Deep Dive into LLMs like ChatGPT"** (3.5 hr, Feb 2025). The full stack: pretraining, fine-tuning, RLHF, why hallucinations happen. Find it on his YouTube channel.
4. **Boris Cherny on Lenny's Podcast** (~90 min). Head of Claude Code on product philosophy. Watch this before you try to use Claude Code seriously. Also find his Latent Space appearance for the technical version.

Do (hands-on, ~5 hours)

Tutorials you actually run.

1. **Anthropic's Prompt Engineering Interactive Tutorial** (free, 9 chapters of Jupyter notebooks, ~4 hr). The best free prompt engineering course that exists. Do every exercise. This is the single highest-ROI thing on this page.
2. **Code w/ Claude: Prompting 101** (Anthropic applied team, 45 min). Real production prompting patterns, not toy examples.
3. **Read the Claude docs.** Start with Agents and Tool Use and Model Context Protocol. Skim them now, come back when you're stuck.

Optional depth

Not required. Worth it if curious.

- **Karpathy, Let's build GPT from scratch** (2 hr). Codes GPT live. The highlight of his Zero to Hero series.
- **Karpathy, Neural Networks: Zero to Hero** (7 videos, ~15 hr). Builds neural nets from scratch: micrograd → makemore → nanoGPT. If you want internals, the best playlist ever made.
- **3Blue1Brown, But what is a GPT?** and Attention in transformers (~50 min together). Animated visual explainers. For the "I'm not a math person but I want intuition" case.
- **Simon Willison's talks**. Running commentary on LLMs as a working tool.

Skip

Be ruthless. Wrong-era resources cost more than they teach.

- **LangChain courses.** AIOMs work on agent platforms, not LangChain.
- **fast.ai Practical Deep Learning.** Great course, wrong audience. It's for ML researchers, not AIOMs.
- **Hugging Face LLM Course.** Fine-tuning and deployment aren't AIOM-core.
- **Most Lex Fridman AI episodes.** Occasional gems, mostly length.
- **Anything from pre-2023 on practical LLMs.** The field has moved twice since. Theory ages well. Practical takes expire in 18 months.

> **Four-week path.** Week 1: Intro to LLMs + Software Is Changing + Prompt Eng Tutorial (first 3 chapters). Week 2: finish the tutorial + Code w/ Claude: Prompting 101 + read Agents/Tool Use docs. Week 3: Deep Dive into LLMs + Boris Cherny on Lenny's. Week 4: start building.
>
> You don't need to finish every video before you build. You learn faster by building halfway through.

CHAPTER 02 · THE MOST IMPORTANT CHAPTER

Build

Nine real projects, sorted so each one teaches something the next depends on.

Theory is table stakes. Podcasts are background radiation. **Projects are the thing.**

If you do nothing else, do this ladder. By rung 7 you can hold your own in any applied-AI interview.

> **The Rule.** Ship every project publicly. GitHub repo + 90-second Loom + LinkedIn post. The artifacts ARE your portfolio. A resume with "interested in AI" is worth nothing. A GitHub profile with nine shipped AI projects is worth a job.

---

1 Automate one annoying task with Claude Code

DIFFICULTY 1 · 2-4 HOURS

Pick one friction point you actually hate: inbox triage, meal planning from fridge photos, weekly calendar summary, workout logging. Install Claude Code, describe the task, iterate until it works.

Teaches: how agents behave, why context matters, prompts as specifications. Easy entry and a preview of how you'll spend days as an AIOM.

---

2 Build a chatbot with the Claude API

DIFFICULTY 1 · ONE WEEKEND

Single-file Python script hitting the Claude API with a system prompt (a persona). Use the Anthropic quickstart.

Teaches: API calls, tokens, system vs. user messages. Removes the abstraction so you see what's actually happening behind every agent builder.

3 RAG over your own notes

DIFFICULTY 2 · ONE WEEKEND

Start easy: drag your journal, Notion export, or Obsidian vault into Claude Projects and chat with it. Then rebuild from scratch with LlamaIndex or the Anthropic RAG cookbook.

Teaches: embeddings, chunking, and the truth that retrieval quality is the whole ballgame. You need to explain RAG to a non-technical stakeholder in 90 seconds. Build one, and you can.

4 Build a tool-use agent

DIFFICULTY 2 · ONE WEEK

Claude with the tool_use API. A bot that reads your calendar, checks weather, and texts you a day plan. Or a finance agent that reads Plaid transactions and yells at you about your grocery bill.

Teaches: function calling, agent loops, and (most importantly) why agents fail. Ambiguous tools, bad state handling, retry strategies, when to ask a human. The meat of the AIOM job.

5 Install MCP servers and write a custom one

DIFFICULTY 2 · A FEW HOURS

Install two or three MCP servers in Claude Desktop: Gmail, Google Calendar, Plaid. Then write a tiny custom MCP server using Anthropic's Python template. Point Claude at something weird in your life.

Teaches: what MCP is and why it's the direction the industry is going. Table stakes now; a floor in six months.

6 Build an agent on a no-code platform

DIFFICULTY 1 · 2-4 HOURS

Glean Agent Builder if you're at a Glean customer. Otherwise Dify, Flowise, LangFlow, or n8n's AI nodes. Build a three-step agent with retrieval, an LLM step, and a tool call.

Teaches: what "building an agent" looks like when the platform abstracts the API. Exactly what AIOMs do with customers all day.

7 Write a simple eval for one of your agents

DIFFICULTY 3 · HALF A DAY

Pick an agent you already built. Write 10-20 test cases with expected outputs. Run them. Score pass/fail. Change one thing in the prompt. Score again.

Teaches: you cannot ship what you cannot measure. The skill that separates hobbyists from people who get hired. See Anthropic's evaluation framework guide.

8 Deploy a public chatbot with the Vercel AI SDK

DIFFICULTY 2 · ONE WEEKEND

Clone vercel/chatbot, customize, deploy to a public URL. Get 10 friends to use it. Watch the logs.

Teaches: streaming, React hooks, and (crucially) shipping to real users. The gap most tutorials skip.

9 Make Claude Code your daily driver

DIFFICULTY 3 · ONGOING

Custom slash commands. Rules files. Skills. MCPs. Replace five manual workflows with Claude-Code-driven ones. Commit your setup to a repo.

Teaches: all of it. The project AIOM interviewers want to see. See Claude Code for the setup.

Optional depth

For the curious, not AIOM-core.

- **Karpathy's makemore + nanoGPT** (Difficulty 4, 1-2 weeks). Watch Zero to Hero, code along, train a tiny model on your own writing. Credibility-bank.
- **Fine-tune a small model** (Difficulty 3, one week). Gemma 3 or Phi-3 on Colab free tier. Teaches data quality beats model size.
- **Voice app: Whisper + Claude + ElevenLabs** (Difficulty 3, one week). Record, transcribe, reason, speak. Teaches multi-model pipelines.

Ideas if you want more

- Home-finance agent that categorizes Plaid transactions and flags weird spending.

- "What did I do this week" agent that summarizes your git commits, calendar, and journal.
- Personal health tracker that ingests Oura data, lab results, and meal photos.
- Slack bot that reads your team's channels and drafts a weekly digest.
- Agent that watches your Gong or Granola transcripts and drafts follow-up emails.

> **One rule before you start.** Pick the smallest version you can finish this week. Shipped and ugly beats unshipped and beautiful. You learn 10x more from one deployed project than from three half-built ones.
>
> **After you ship.** GitHub README → 90-second Loom → LinkedIn post → tweet it. Simon Willison, Ethan Mollick, and Alex Albert actually reply to people shipping good work. Ship and they'll find you.

CHAPTER 03 · THE FULL SETUP WALKTHROUGH

Claude Code

Build your AI agent. A persistent brain Claude can read, write to, and auto-sync to GitHub.

Claude Code is Anthropic's CLI. Type plain English and it writes code, edits files, runs commands, and hands back results. Better than ChatGPT for anything touching your actual filesystem. Single biggest productivity unlock I've found in two years.

This chapter is a full practical walkthrough of the setup I actually run. By the end you'll have a `~/brain/` repo that Claude reads on every session, auto-synced to GitHub every 10 minutes, with skills and rules that make it yours.

Why This Setup

Every Claude Code session starts fresh. No memory of yesterday. The fix: give it a persistent file system it can read on startup and write to during work. The Brain is a private GitHub repo cloned to `~/brain/`. Claude reads it for context, writes to it after doing work, and a cron job syncs everything to GitHub every 10 minutes. That's the whole system.

The result: Claude knows who you are, what you're working on, which projects you manage, what happened last session, and what mistakes it made last time. It picks up where it left off every single time.

Everything is markdown. No databases, no config files, no infrastructure. Markdown is what Claude reads and writes best, it's version-controlled, diffable, and you can open any file and understand it without tools.

Why this structure works:

- **Separation of concerns.** Work context (`work/`) is separate from system context (`memory/`), which is separate from automation (`skills/` , `scripts/`). Claude knows exactly where to look.
- **Customer (or project) folders as units of context.** Each folder has a consistent shape: README (what's true now), log.md (what happened over time), dated files (working outputs). Claude reads the folder and is immediately caught up.
- **Brain is canonical.** Files live in `~/brain/` and get symlinked to `~/.claude/` . Never edit the `~/.claude/` copies directly. One source of truth.

What You're Building

```
~/brain/                    # Your private GitHub repo (source of truth)
  CLAUDE.md                 # Who you are, how Claude should work with you
  todos.md                  # Running task list Claude can read and update
  memory/                   # Claude's self-improving notes
    MEMORY.md               # Index file (auto-loaded every session)
    tool-lessons.md         # What went wrong, how to fix it
    preferences.md          # Your feedback on Claude's output
  work/                     # Your work context (customers, projects, notes)
  skills/                   # Reusable workflows Claude can execute
  commands/                 # Quick-fire templates
  scripts/
    auto-sync.sh            # Cron job: commits and pushes every 10 min
    setup-claude.sh         # One-time wiring script
  .github/workflows/        # Recurring automations (digests, health reports)
```

The key insight: Claude Code reads `~/CLAUDE.md` and `~/.claude/` on every session start. Symlink those to a Git repo and your AI agent has persistent memory across sessions and machines.

Step 1: Get Claude Code Running

INSTALL

```
# Via npm (requires Node.js 18+)
npm install -g @anthropic-ai/claude-code

# Or Homebrew
brew install claude-code
```

SUBSCRIBE

- **Claude Pro** ($20/mo): limited usage.
- **Claude Max 5x** ($100/mo): recommended. Plenty of headroom.
- **Claude Max 20x** ($200/mo): heavy usage.

FIRST RUN

```
cd ~
claude
# Walks you through authentication
```

Step 2: Create Your Brain Repo

INITIALIZE

```
mkdir ~/brain && cd ~/brain
git init
mkdir -p memory work skills commands scripts notes reference .github/workflows
```

CREATE YOUR CLAUDE.MD

The most important file. It tells Claude who you are and how to work with you. Drop this at `~/brain/CLAUDE.md` and customize:

```
# [Your Name]

## Communication
- [Tone: concise, detailed, casual, formal]
- [Rules: no jargon, no emojis, no em dashes, etc.]
- [Push back when warranted? Or always agree?]

## Work
- [Your role, company, team]
- [What you're working on right now]
- [Work email vs personal email]

## File System
- **Brain is source of truth.** Write to `~/brain/` first.
- Todos: `~/brain/todos.md`
- Memory: `~/brain/memory/`
- Work context: `~/brain/work/`
- Save generated files to `~/Downloads`

## Session Rules
- Read relevant context files before starting work on a topic
- Update todos.md when new tasks come up
- Update memory files when you learn something about tools or preferences
- Ask on ambiguous requests. Act on clear tasks.
```

SEED THE MEMORY FILES

```
cat > ~/brain/memory/MEMORY.md << 'EOF'
# Claude Memory

## Key Facts
- [Populated as Claude learns about you]

## Tool Lessons
- See tool-lessons.md

## Preferences
- See preferences.md
EOF

printf "# Tool Lessons\n\nLog tool failures and correct approaches here." > ~/brain/memory/tool-
printf "# Preferences\n\nLog feedback on format, tone, and approach here." > ~/brain/memory/pref
printf "# Debugging\n\nLog recurring issues and solutions here." > ~/brain/memory/debugging.md
printf "# Todos\n\nClaude can read and update this file across sessions." > ~/brain/todos.md
```

Step 3: Wire Claude Code to Your Brain

SYMLINK CLAUDE.MD

Claude Code looks for `~/CLAUDE.md` on every session start:

```
ln -sf ~/brain/CLAUDE.md ~/CLAUDE.md
```

WIRE CLAUDE'S MEMORY DIRECTORY

```
MEMORY_DST="$HOME/.claude/projects/-Users-$(whoami)/memory"
mkdir -p "$MEMORY_DST"
cp ~/brain/memory/*.md "$MEMORY_DST/"
```

SYMLINK SKILLS AND COMMANDS

```
[ -d ~/.claude/skills ] && mv ~/.claude/skills ~/.claude/skills.bak
[ -d ~/.claude/commands ] && mv ~/.claude/commands ~/.claude/commands.bak

ln -sfn ~/brain/skills ~/.claude/skills
ln -sfn ~/brain/commands ~/.claude/commands
```

SAVE AS A ONE-SHOT SETUP SCRIPT

Save as `~/brain/scripts/setup-claude.sh` so you can re-run it on any machine:

```
#!/bin/bash
set -e

BRAIN_DIR="$HOME/brain"
CLAUDE_DIR="$HOME/.claude"
MEMORY_DST="$CLAUDE_DIR/projects/-Users-$(whoami)/memory"

ln -sf "$BRAIN_DIR/CLAUDE.md" "$HOME/CLAUDE.md"

for sub in skills commands; do
  if [ -d "$CLAUDE_DIR/$sub" ] && [ ! -L "$CLAUDE_DIR/$sub" ]; then
    mv "$CLAUDE_DIR/$sub" "$CLAUDE_DIR/$sub.bak"
  fi
  ln -sfn "$BRAIN_DIR/$sub" "$CLAUDE_DIR/$sub"
done

mkdir -p "$MEMORY_DST"
cp "$BRAIN_DIR"/memory/*.md "$MEMORY_DST/" 2>/dev/null || true

echo "Done. Claude Code is now using Brain as source of truth."
```

Step 4: Auto-Sync to GitHub

Keeps your brain backed up and accessible from any machine. A cron job commits and pushes every 10 minutes.

PUSH TO GITHUB

```
cd ~/brain
gh repo create Brain --private --source=. --remote=origin
git add -A && git commit -m "initial brain setup"
git push -u origin main
```

CREATE AUTO-SYNC.SH

Save as `~/brain/scripts/auto-sync.sh`:

```
#!/bin/bash
cd ~/brain || exit 1

if [[ -d .git/rebase-merge || -d .git/rebase-apply ]]; then
    echo "$(date '+%Y-%m-%d %H:%M') - Rebase in progress, skipping"
    exit 0
fi

STASHED=false
if [[ -n $(git status --porcelain) ]]; then
    git stash -q && STASHED=true
fi

git fetch origin main -q
if ! git rebase origin/main -q 2>/dev/null; then
    git rebase --abort 2>/dev/null
    $STASHED && git stash pop -q 2>/dev/null
    echo "$(date '+%Y-%m-%d %H:%M') - Rebase conflict"
    exit 1
fi

$STASHED && git stash pop -q 2>/dev/null

if [[ -n $(git status --porcelain) ]]; then
    git add -A
    git commit -q -m "auto-sync $(date '+%Y-%m-%d %H:%M')"
fi

[[ -n $(git log origin/main..HEAD --oneline) ]] && git push origin main -q
```

INSTALL AS A CRON JOB

```
chmod +x ~/brain/scripts/auto-sync.sh

(crontab -l 2>/dev/null; echo "*/10 * * * * $HOME/brain/scripts/auto-sync.sh >> $HOME/brain/scri
```

Step 5: Add MCP Servers

MCP (Model Context Protocol) servers give Claude access to external tools: calendar, email, documents, search. Configure in `~/.claude/settings.json`:

```
{
  "mcpServers": {
    "search": {
      "command": "npx",
      "args": ["-y", "@anthropic-ai/mcp-server-web-search"]
    }
  }
}
```

Worth installing: Google Calendar, Gmail, GitHub, Plaid, Granola (meeting transcripts). If you're at a company with Glean, the Glean MCP is where the real magic happens for work context: Slack, email, docs, transcripts, Salesforce, all in one. Full directory at modelcontextprotocol.io.

Step 6: Build Your First Skill

Skills are markdown files that teach Claude to do a specific task. They live in `~/brain/skills/` and auto-load via the symlink.

EXAMPLE: MEETING PREP

`~/brain/skills/meeting-prep/SKILL.md`:

```
# Meeting Prep

Prepare a briefing for an upcoming meeting.

## Trigger
When the user says "prep me for [meeting]" or "brief me on [account]"

## Steps
1. Check local files at ~/brain/work/ for existing context
2. Search recent communications and documents on the topic
3. Look up calendar for attendees and agenda
4. Synthesize into a brief:
   - Key context and background
   - Open action items
   - Suggested talking points
   - Risks or things to watch for

## Output
Concise bullets. No fluff. Lead with the most important thing.
```

EXAMPLE: QUICK LOG

`~/brain/skills/log/SKILL.md`:

```
# Quick Log

Capture an activity note.

## Trigger
/log [topic] [what happened]

## Steps
1. Parse the topic and content
2. Append a dated entry to the relevant file in ~/brain/
3. If new action items were mentioned, add them to ~/brain/todos.md

## Format
## YYYY-MM-DD
- [bullets of what happened]
```

Step 7: Rules Files

Rules live in `~/.claude/rules/` and load on every session. Use them for behavioral instructions that should always apply.

EXAMPLE: `~/.CLAUDE/RULES/WORK-CONTEXT.MD`

```
# Work Context

## My Role
- [Your role and day-to-day]
- [Key projects or accounts]

## How I Work
- I prefer [concise / detailed] responses
- Always [read context first / just dive in]
- When I say "remember this," add it to ~/brain/notes/ or ~/brain/todos.md
- When I mention a task I need to do, add it to ~/brain/todos.md
```

EXAMPLE: `~/.CLAUDE/RULES/SESSION-RULES.MD`

```
# Session Rules

- Read relevant context files before starting work on a topic
- After finishing substantive work, update the relevant log file
- Parallelize independent lookups when possible (use subagents)
- Ask before pushing to GitHub
```

How It All Fits Together

```
You open a terminal and type: claude

Claude Code starts up and automatically reads:
  1. ~/CLAUDE.md              → knows who you are
  2. ~/.claude/rules/*.md     → knows your preferences
  3. ~/.claude/.../MEMORY.md  → remembers past sessions

You say: "prep me for my meeting with Acme Corp"

Claude:
  - Reads ~/brain/work/customers/acme-corp/ for context
  - Uses MCP tools to search recent emails, docs, transcripts
  - Synthesizes a brief with talking points

After the meeting, you say: "log that we discussed X, Y, Z.
Action item: send proposal by Friday"

Claude:
  - Appends a dated entry to ~/brain/work/customers/acme-corp/log.md
  - Adds "Send Acme proposal by Friday" to ~/brain/todos.md

10 minutes later, auto-sync commits and pushes to GitHub.

Next session, maybe on a different machine, Claude picks up
right where you left off.
```

Quick Reference

WHAT	WHERE
Master config	`~/brain/CLAUDE.md` (symlinked to `~/CLAUDE.md`)
Behavioral rules	`~/.claude/rules/*.md`
Memory (auto-loaded)	`~/.claude/projects/-Users-YOU/memory/MEMORY.md`
Memory (canonical)	`~/brain/memory/`
Todos	`~/brain/todos.md`
Skills	`~/brain/skills/` (symlinked to `~/.claude/skills/`)
Commands	`~/brain/commands/` (symlinked to `~/.claude/commands/`)
Auto-sync	`~/brain/scripts/auto-sync.sh` (cron, every 10 min)
Setup script	`~/brain/scripts/setup-claude.sh` (run once per machine)
GitHub Actions	`~/brain/.github/workflows/`

Useful Commands

```
# Start a new session
claude

# Continue your last session
claude --continue

# Rename the current session (easier to find later)
/rename meeting-prep-acme

# Check auto-sync logs
tail -20 ~/brain/scripts/sync.log
```

Tips

1. **Start small.** Get CLAUDE.md and the brain repo working first. Add skills and MCPs as you find patterns worth automating.

2. **Let Claude update its own memory.** When a tool fails or you correct its output, it should log that in the memory files so it doesn't repeat the mistake.
3. **Markdown for everything.** Claude reads and writes markdown natively. Version-controlled, diffable, human-readable.
4. **Parallelize.** If you need five independent lookups, Claude can spin up subagents instead of going sequential.
5. **Build skills for repetitive work.** Same type of task twice? Write a skill.
6. **CLAUDE.md is a living document.** Update it as your role, projects, and preferences change. The more accurate it is, the better Claude performs.

> **Warning.** Claude Code can do destructive things. Don't give it unsupervised access to production databases. Don't let it `rm -rf` blindly. Always confirm before a `git push`.

CHAPTER 04

What to Follow

Podcasts, blogs, and X accounts worth your time.

Pick 2 podcasts, 2 newsletters, and a dozen X follows. Consistency beats breadth. If you try to follow everyone, you follow no one. The goal is calibrated intuition, not winning a Jeopardy category.

Podcasts (actually in my Spotify)

AI-FOCUSED

AI Daily Brief	Nathaniel Whittemore. 10 minutes a day. Best signal-to-noise for AI news. My morning default.
TBPN	Tech, markets, AI. Go for the shortened versions. The full episodes run long.
No Priors	Sarah Guo + Elad Gil. VC-flavored, technically sharp. Guests run the industry.
Training Data	Sequoia. Short, founder-focused AI interviews.
Latent Space	swyx + Alessio. The builder's show. Infra, evals, agent frameworks.
How I AI	Claire Vo. How people actually use AI in their work, week by week.

BROADER TECH ROTATION

Stratechery	Ben Thompson. Strategy takes on tech and AI. Also the paid newsletter below.
All-In	Chamath, Jason, Sacks, Friedberg. Skip heavy politics; the tech/markets takes are the reason.
Lenny's Podcast	PM + career. The Boris Cherny episode on Claude Code is a must.
BG2Pod	Brad Gerstner + Bill Gurley. Tech investing, strategic.
Y Combinator Podcast	Founder-focused.
This Week in Startups	Jason Calacanis.
Moonshots	Peter Diamandis. Future tech, optimism-heavy.
My First Million	Entrepreneurship.

WORTH ADDING FOR DEPTH

Dwarkesh Podcast	Long, unhurried interviews with the people actually building frontier AI. Dario, Hassabis, Ilya, Karpathy. I don't currently listen; you should.

Reading

WHAT I PAY FOR

Stratechery	Ben Thompson, paid. Clearest strategic thinking on what AI means for incumbents, platforms, and power. Worth every dollar.

WORTH ADDING

Simon Willison's weblog	Free. The single best daily AI writer working. Ships tools, tests every model, links everything that matters.
Import AI	Jack Clark, Anthropic cofounder. Weekly. Policy, capability jumps, strategic framing from inside the frontier.

X

The dozen I actually follow, plus two official accounts.

@karpathy Teaches more in a thread than most courses.	**@simonw** Live-tweets every model release with tests.	**@dwarkesh_sp** Podcast clips and takes.
@DarioAmodei Anthropic CEO. Rare but signal-heavy.	**@sama** OpenAI CEO. Watch the vibes, not the words.	**@nlw** Nathaniel Whittemore, AI Daily Brief host.
@swyx Latent Space, builder scene.	**@AlexAlbert__** Prompt craft and Claude tips.	**@jackclarkSF** Policy and capability reasoning.
@benthompson Stratechery.	**@rileybrown** Vibe coder, practical AI user.	**@alexfinn** Vibe coder, practical AI user.
@AnthropicAI Official announcements.	**@OpenAI** Official announcements.	

Skip crypto, culture war, and anyone who makes you feel worse after reading them. Your attention is the most valuable input you have.

How I actually use this

MORNING	AI Daily Brief on the subway. 10 minutes.
COMMUTE	TBPN shortened versions. All-In or BG2Pod on long flights.
WEEKLY	Stratechery when Ben ships a big one.
AMBIENT	X feed, tightly curated. Mute generously.

> *Consistency beats volume. If you're consuming more than you're building, your ratio is off.*

CHAPTER 05

The Role

What AIOMs actually do, and what skills you need to get there.

You're not going to become an AI researcher at Anthropic without a PhD. Fine. The most valuable roles in AI right now are not research roles. They're the people between a research lab and a paying customer: **Forward Deployed Engineers, Applied AI Engineers, Customer Engineers, AI Outcomes Managers**. They build prototypes against real customer data, run enablement, turn ambiguous pain into shippable agents, and feed what they learn back into the product.

A typical AIOM week

A real week. Not polished. Representative.

MONDAY	Build a campaign-brief-generator agent for a customer's marketing team that pulls from Snowflake. 90 minutes use-case discovery, 90 prompt iteration, 60 testing.
TUESDAY	Roll out an agent champion program at a 3,000-person enterprise. Train 15 department leads. Write a best-practices doc.
WEDNESDAY	Help another AIOM unblock their customer. Post 500 words in #aiom-team on a technical pattern. Interview a candidate.
THURSDAY	Executive briefing at a Fortune 500. Present agent portfolio, usage numbers, 90-day plan. Schedule the next six weeks of work.
FRIDAY	Ship a new Claude Code skill to automate part of your own job. Run a 160-person webinar on prompt engineering. Log everything.

You are a hybrid consultant / prompt engineer / product thinker. Scored on **adoption** (is the customer actually using it?) and **power agents** (are the agents still used weekly, months later?). Both metrics reward learn-by-doing. Neither rewards talk.

How you get there

The whole guide is the answer. A summary:

1. **Build in public.** Every project: GitHub repo, 90-second Loom, LinkedIn post. Your visible work becomes the resume.
2. **Cold outreach is a working demo.** Ship something on the target company's API. 90-second walkthrough. Send it with one specific question. Reply rate is 10x generic outreach.
3. **Pick a vertical and go deep.** Domain expertise plus AI fluency is rarer than either alone.

4. **Write.** A Substack with five posts of "here is what I built and what I learned" outperforms any polished resume.
5. **Skip the bootcamps.** Reforge or Refonte only buys you a cohort you can replicate with a Discord server.
6. **Treat the job search itself as a project.** Build an agent that finds roles, drafts tailored cover letters, tracks applications. Show it to your interviewers.

THE SINGLE PIECE OF META-ADVICE

The gap between *one shipped* project and *zero* is wider than any credential in the industry right now.

Closing that gap is a weekend of work. So go.

Harry Toig
AI Outcomes Manager, Glean
github.com/harrytoig/nicks-guide-to-ai